

REPORT

Training/AutoResearch 노드 — 통합 보고서

한 줄: 단일 GPU(RTX 4090)에서 growing-memory 설계공간을 자동 탐색(AutoResearch)하는 노드를 Pi AI Agent 웹 서비스로 구현하고, **메모리캐싱(선형/Titans/DeltaNet)** 추론의 **효율**과 **정확도 트레이드오프**를 4090 실측으로 검증했다. 모든 수치는 재현 가능(스크립트 명시).

저장소: <https://github.com/hongsw/pi-agent-webservice-kit> · 실험 노드: RTX 4090 24GB, torch 2.10+cu128.

1. 무엇을 만들었나 (과제 4요소 + 설계)

- **Pi Agent:** AutoResearch 오케스트레이터(`tutorial/autoresearch/`) — 게이트→ASHA→리더보드→best.
- **Skill x2:** validity-gate(학습 전 동치/shape), leaderboard-analysis(best·프록시 신뢰도).
- **MCP:** `autoresearch-mcp` 7도구(리더보드/NAS/run). **Pi Extension:** sweep start/stop·export.
- **Web UI:** 무설치 stdlib 대시보드. **도커:** NAS·Edge 서버(`docker/`).
- karpathy/autoresearch 원형(ratchet: train.py 편집→val_bpb→keep/revert)을 제조+growing-memory로 일 반화.

2. 핵심 알고리즘 효과 — 메모리·계산량 효율 (검증된 구현으로 재측정)

효율 벤치 (deltanet[fla] vs 트랜스포머[FlashAttention], d256/L4, bf16, batch1; `efficiency_bench.py`) | L
| DeltaNet ms / GB | TF ms / GB | TF/DN 시간 | |---|---|---|---| | 8192 | 3.8 / 0.46 | 2.1 / 0.23 | 0.55× | | 32768
| 6.9 / 0.55 | 18.5 / 0.42 | 2.7× | | 65536 | 14.8 / 0.67 | 65.4 / 0.67 | 4.4× | | 131072 | 31.2 / 1.15 | 244 / 1.17 |
7.8× |

- **계산량:** DeltaNet $O(L)$, 트랜스포머 $O(L^2)$ (FlashAttention은 *메모리만* 선형, 연산은 제곱). → 긴 컨텍스트에서 DeltaNet이 **시간 8× 빠름**(128K). 교차점 ~16-32K.
- **메모리(학습/prefill):** 둘 다 $O(L)$, 거의 동일(둘 다 chunked/flash).
- **추론 메모리:** 선형/Titans/DeltaNet은 $O(1)$ 상수 상태 vs 트랜스포머 KV캐시 $O(L)$. 실측(소형, `infer_bench.py`): 컨텍스트 무관 재귀상태 **132KB** vs KV 1M=9.7GB·2M=19GB·4M OOM → 1×4090+선형 이 16M=2×A100, 64M=7×A100, 256M=26×A100의 KV 용량 대체($\approx 992\times @16384$).
- **단일 4090 chunked 학습 최대 컨텍스트:** **131,072(128K)** 토큰($O(L)$, `maxseq.py`).

정정 이력(정직): 초기 from-scratch 선형 구현은 $L\times L$ 을 materialize해 $O(L^2)$ ·16384 OOM이었음(버그). chunked 재구현으로 $O(L)$ 해결. 위 표는 **검증된 fla deltanet** 기준(naive 버그 없음).

3. 정확도(품질) — grok과 트레이드오프

MQAR 연관회상으로 측정. 메모리 절감은 공짜가 아니며, 적절한 구현+학습으로 동급 정확도 달성 가능.

- **vetted 구현 검증**(단일 LayerNorm 스캐폴드, `fla_validate.py`): `deltanet/gated_deltanet/retention/titans = recall 1.0`. `linear(fla)=0.54`(선형 회상 약점), `gla=0.06`(튜닝 필요).
- **AutoResearch 연결 검증**: `deltanet`을 `base_rule`로 연결 → 하니스에서 **recall 1.0(grok@1000)**.
- 긴 컨텍스트: `titans`(자체) `seq=512`에서 **1.0(grok@15k)** — 완벽회상+O(1) 추론 동시.
- `grokking` = 정체 후 급점프(Power et al. 2022). 단 무한데이터라 엄밀히는 학습 중 상전이.

4. 정확 구현 우선 (교훈 반영)

- "growing-memory"는 단일 라이브러리가 아니라 **설계 4축 공간**. 정확 구현 = `vetted` 라이브러리: `fla`(`LinearAttention/GLA/DeltaNet/GatedDeltaNet/MultiScaleRetention`) + `titans-pytorch`(`lucidrains`).
- 자체 from-scratch `titans`는 chance로 실패 → `lucidrains` 채택. (`TITANS.md`)
- 순서: **정확 구현** → **단일 스캐폴드 검증(sanity+recall)** → **autoresearch 연결**. (`FLA_VALIDATION.md`)

5. pip 패키지 & 통합

`growing-memory-pytorch` (`packages/growing-memory`): 코어 + HF/Unsloth 래퍼
(`GrowingMemoryForCausalLM`)

- `fla/titans` 백엔드. `AutoResearch`가 `import growing_memory` 로 실물 백엔드 사용. `pip install -e "...[hf,titans]"`.

6. 실 데이터 (HC-SR04)

실 초음파 거리센서 데이터로 §5 "예측오차 이상탐지" 실증(`HCSR04.md`): 선형보정 $R^2=0.998$, 드리프트 탐지(-22.8 SE), 주입 이상치 recall 0.95. (단 900행 보정셋, 드리프트는 단일 레벨시프트 — 한계 명시)

7. 한계 & 다음

- `gla/retention` 풀 하니스 `grok` 폴리시(스캐폴드 민감). 실 NAS 데이터·1.3B 스케일 미수행(인터페이스만).
- 효율 우위는 **추론(O(1) 상수메모리) + 긴 컨텍스트 연산(O(L))**에서 결정적; 단기/소컨텍스트는 트랜스포머 유리.
- 트레이드오프(효율↔정확도)를 데이터별로 찾는 것이 `AutoResearch` 노드의 존재 이유.

근거 문서 (재현)

[COMPARE3](#) · [INFERENCE](#) · [LONGSEQ](#) · [GROKING](#) · [RECURRENT](#) · [TITANS](#) · [FLA_VALIDATION](#) · [IMPACT](#) · [HCSR04](#) · [RESULTS](#)